



**ALLIANCE
UNIVERSITY**
Private University established in Karnataka State by Act No. 34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi.

**NAAC
GRADE A+**
ACCREDITED UNIVERSITY

Bachelor of Computer Applications

Semester – V

Deep Learning

Experiment No.: 7

Title: Loss Function Implementation

Submitted By

Name: Dipin Roka

Reg. No.:2411021240007

Date:29/09/2026

[GitHub Repo](#)

Submitted To:

Faculty Name: Nagasri p

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal Bengaluru – 562106

Assignment: Understanding Loss Functions in Neural Networks

Objective

To understand how neural networks measure prediction errors by implementing Mean Squared Error (MSE) and Binary Cross-Entropy (BCE) from scratch using NumPy, and comparing actual and predicted values.

Introduction

A **loss function** tells a neural network how wrong its predictions are. It compares the actual value with the predicted value and gives one number. A high loss means bad predictions, and a low loss means good predictions. During training, the network tries to reduce this number by adjusting its weights. Different problems need different loss functions. MSE is mostly used for regression, and Cross-Entropy is mostly used for classification.

Task 1: Dataset (Actual vs Predicted)

Sample	Actual (y)	Predicted (ŷ)	probability	• Actual labels are the true classes: 1 = positive class (e.g., spam) and 0 = negative class (e.g., not spam).
1		0.9		
0		0.2		
1		0.7		• Predicted values are the model's outputs, usually from a sigmoid function. Each is the probability that the sample belongs to class 1. For example, 0.9 means the model is 90% sure the sample is class 1.
1		0.6		
0		0.1		

```
import numpy as np

# Actual labels (ground truth) and predicted probabilities from the model
y_actual = np.array([1, 0, 1, 1, 0]) # true labels
y_predicted = np.array([0.9, 0.2, 0.7, 0.6, 0.1]) # predicted probabilities

print("Actual values      :", y_actual)
print("Predicted values:", y_predicted)
```

Task 2: Loss Functions from Scratch

$$\text{MSE} = (1/n) \cdot \sum_i (y - \hat{y})^2$$

$$\text{BCE} = -(1/n) \cdot \sum_i [y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

A small epsilon is used so that $\log(0)$ never happens.

```

def mean_squared_error(y_true, y_pred):
    """MSE = (1/n) * sum((y - y_hat)^2)"""
    squared_errors = (y_true - y_pred) ** 2    # square each
    error          return np.mean(squared_errors)    # average
    of squared errors

def binary_cross_entropy(y_true, y_pred):
    """BCE = -(1/n) * sum(y*log(y_hat) + (1-y)*log(1-y_hat))"""
    epsilon = 1e-15    # tiny value
    to avoid log(0)
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)    # keep
    values inside (0, 1)
    losses = y_true * np.log(y_pred) + (1 - y_true) * np.log(1 -
    y_pred)
    return -np.mean(losses)    # negative
    average

```

Task 3: Calculate and Compare the Losses

```

mse = mean_squared_error(y_actual,
y_predicted) bce =
binary_cross_entropy(y_actual, y_predicted)

print("--- Loss Values ---")
print("Mean Squared Error (MSE)  :", round(mse, 4))
print("Binary Cross-Entropy (BCE):", round(bce, 4))

# Extra experiment: make sample 3 a confident wrong prediction
(actual
= 1, predicted = 0.01)
y_predicted_bad = np.array([0.9, 0.2, 0.01, 0.6, 0.1])

print("--- With one confident wrong prediction (sample 3: 0.01) -
---") print("MSE:", round(mean_squared_error(y_actual,
y_predicted_bad), 4)) print("BCE:",
round(binary_cross_entropy(y_actual, y_predicted_bad), 4))

```

Expected Output

```

Actual values    : [1 0 1 1
0]
Predicted values: [0.9 0.2 0.7 0.6
0.1]
--- Loss Values -
--
Mean Squared Error (MSE)  :
0.062

```

```

Binary Cross-Entropy (BCE): 0.2603
--- With one confident wrong prediction (sample 3: 0.01)
---
MSE:
0.24
BCE:
1.11

```

Step-by-Step Calculation

MSE

Sample	y	$\hat{y}$	$(y - \hat{y})^2$
1	1	0.9	0.01
2	0	0.2	0.04
3	1	0.7	0.09
4	1	0.6	0.16
5	0	0.1	0.01

Sum = 0.31, so MSE = $0.31 / 5 = \mathbf{0.062}$

Binary Cross-Entropy

For $y = 1$ the loss is $-\log(\hat{y})$. For $y = 0$ the loss is $-\log(1 - \hat{y})$.

Sample	y	$\hat{y}$	Term	Loss
1	1	0.9	$-\log(0.9)$	0.1054
2	0	0.2	$-\log(0.8)$	0.2231
3	1	0.7	$-\log(0.7)$	0.3567
4	1	0.6	$-\log(0.6)$	0.5108
5	0	0.1	$-\log(0.9)$	0.1054

Sum = 1.3014, so BCE = $1.3014 / 5 \approx \mathbf{0.2603}$

MSE Explanation

MSE = 0.062 means that, on average, the squared difference between actual and predicted values is 0.062. Squaring makes all errors positive and gives extra weight to larger errors. The value is small because all the predictions are on the correct side of 0.5. The least accurate one is sample 4 (0.6 for an actual 1), which has the largest squared error (0.16).

Cross-Entropy Explanation

BCE ≈ 0.2603 measures how well the predicted probabilities match the true labels. When the model gives a high probability to the correct class, the loss is small. When it gives a low probability to the correct class, the loss grows very quickly. A perfect model would have a BCE of 0. In the extra example, changing one prediction to 0.01 (confidently wrong) raised BCE from

0.26 to 1.11, while MSE only rose from 0.062 to 0.24. This shows that Cross-Entropy punishes confident wrong answers much more strongly.

Why MSE and BCE Should Not Be Compared Directly

MSE (0.062) being smaller than BCE (0.2603) does **not** mean MSE is "better" or that the model is more accurate by that measure. They use different formulas and different scales. MSE measures the squared distance between numbers, while BCE measures the negative log-probability assigned to the correct class. The same predictions give different numbers under each one, and each is only meaningful when compared with values from the same loss function (for example, before and after training).

Which is better for binary classification? Binary Cross-Entropy is usually the better choice because:

- It is designed for probabilities and works naturally with the sigmoid output.
- It penalizes confident wrong predictions heavily.
- It gives stronger gradients when the model is wrong, so learning is faster. MSE with sigmoid can learn slowly because its gradients become very small.

When MSE is still useful:

- Regression problems such as predicting house prices or temperature.
- Cases where the target is a continuous value rather than a class.
- Situations where large numerical errors should be penalized and the output is not a probability.
- Simple baselines and quick experiments.

Comparison Table

Loss Function	Purpose	Common Use	Interpretation	Conclusion
MSE	Measures squared prediction error	Regression	Penalizes larger errors	In this practical, I implemented MSE and Binary Cross-Entropy from
Cross-Entropy	Measures probability/classification error	Classification	Strongly penalizes confident incorrect predictions	

scratch using NumPy and applied them to a small binary classification example. The results were MSE = 0.062 and BCE $\approx$ 0.2603. Both losses show that the model's predictions are fairly good, but they measure error differently, so their values cannot be compared directly. The extra test showed that CrossEntropy reacts much more strongly to a confident wrong prediction. For binary classification, Cross-Entropy is the more suitable loss function, while MSE remains the standard choice for regression problems.